

From coding assistance to agentic engineering

Assist

Ground

Delegate

Customize

Govern

Specify



Inline suggestion



IDE · Agent / Plan



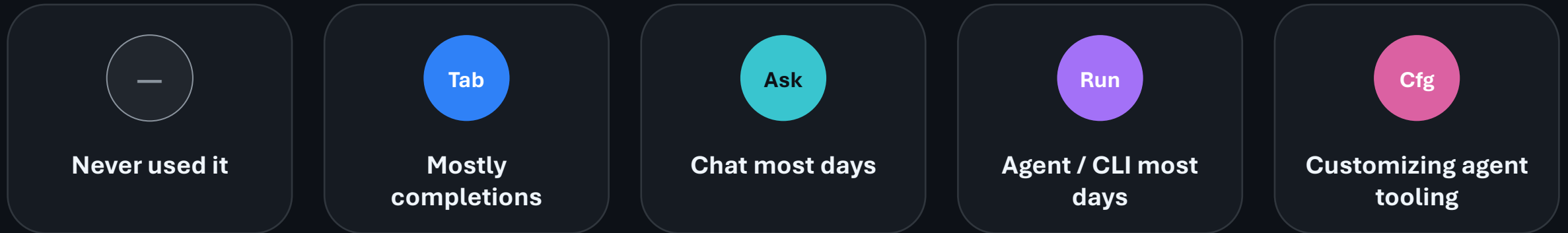
Copilot CLI



Copilot app · parallel sessions

Where are you today?

Live poll — raise a hand for where you are right now.



Goal: move one step forward.

No requirement to use every feature. Use the least complex workflow that solves the task.

One product, three capability layers

Assistive

Chat · inline

Suggest · Explain · Summarize

Agentic

CLI · cloud · app

Plan · Edit · Run · Iterate

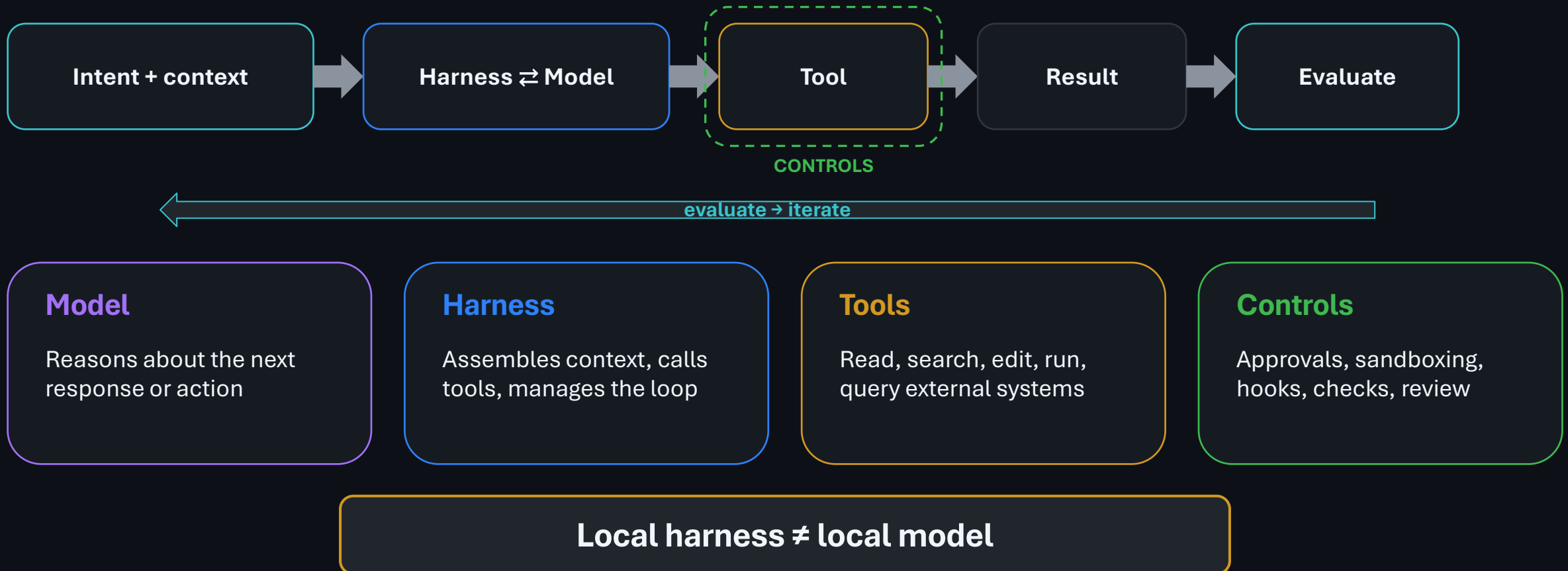
Agent Customization

Instructions · skills · MCP · hooks

Context · Tools · Controls

Synchronous help vs delegated execution. The customization layer is what makes agentic use fit your codebase and team.

Model $\neq$ harness $\neq$ tools



Where the harness runs is separate from where the model runs — that's why IDE, CLI, app, and cloud agent feel different.

Which model? Start with Auto

Auto — one setting, spans all four

Deep reasoning

Architecture / ambiguous debugging / modernization

Coding-focused

Multi-file implementation / tool use / text extraction

Balanced

Everyday explanation, coding, and fixes

Lightweight

Quick, bounded tasks / boilerplate / repetitive edits

Manual routing — choose when:

You know a model is particularly strong for the task

You need the same model for repeatable evaluation

Policy or availability constrains the choice

Speed, context capacity, or tool support matters

Choose manually only for a reason.

Compliance · a known hard task · predictable latency · comparison

Choose the surface by where work should happen

Running example: “Add priority filtering” — same task, different harness		
At the cursor	Inline suggestions	accept a small local completion
In the codebase	IDE Ask / Plan / Agent	scoped, locally verifiable change
In the terminal	Copilot CLI	terminal-first or scripted work
On GitHub	Chat / cloud agent / PR	async, branch-bounded work
Across workstreams	Copilot app	parallel, isolated sessions

Better prompts are better task definitions

Goal

+

Context

+

Constraints

+

Examples

+

Acceptance checks

WEAK PROMPT

Add a priority filter.

Ambiguous — what, where, which values, and how is it tested?

STRONG PROMPT — goal + context + constraints + examples + checks

Add priority filtering to Taskify.

Use `cli.py`, `service.py`, and `test_cli.py`.

Keep the public API backwards compatible.

Accept low, medium, and high; default to all.

Test invalid values and stable ordering.

Run the targeted tests and report evidence.

Context isn't only prose. Files, tests, history, and tools all count. Start a new thread when the task changes.

Context shortcuts worth learning

IDE

`/explain` `/tests` `/fixTestFailure`

slash commands

`#file` `#selection` `@github`

context references

CLI

`Plan` `@path` `/context` `/compact`
`/agent`

`@path` references files · `/compact` trims context · Plan before Agent

 **Rubber duck is an agent — not `/rubber-duck`**

Type `/` or `@` in your current surface for the authoritative list. `/help` is the safest discovery path. Commands are not universal across surfaces.

The customization ladder

Always apply a rule

→ **Instructions**

`.github/copilot-instructions.md` or `.github/instructions/*.instructions.md`

Reuse one task template

→ **Prompt file**

`.github/prompts/*.prompt.md`

Package a workflow

→ **Skill**

`.github/skills/*/SKILL.md`

Specialize a teammate

→ **Custom agent**

`.github/agents/*.agent.md`

Start small

Start with instructions.

Add complexity only when repetition justifies it.

Instructions load automatically.
Prompt files stay supported — manual and surface-limited.

Paths are exact — and no single primitive replaces the others.

Built-in tools, MCP, and deterministic hooks

Built-in tools

Read · search · edit · terminal

in-harness, always present

MCP

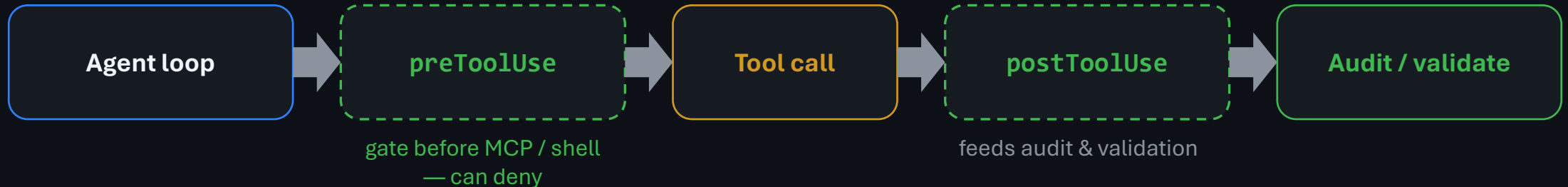
External tools & current data

extends capability, not intelligence

Hooks

Guaranteed lifecycle checks

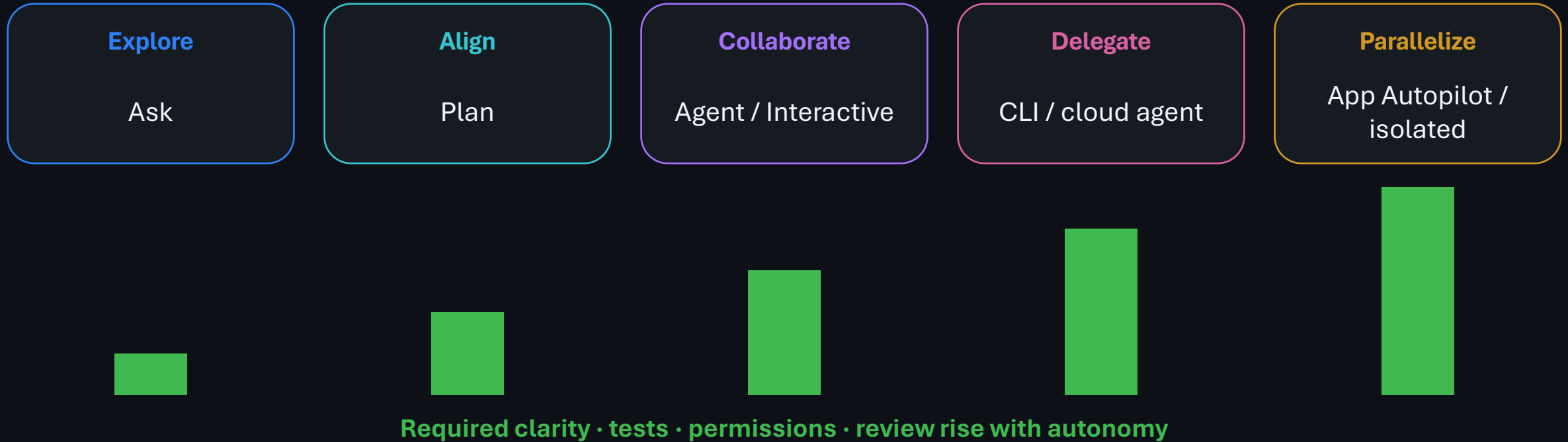
deterministic, not probabilistic



Fewer tools = better selection + lower risk

Autonomy is a dial

AUTONOMY RISES →

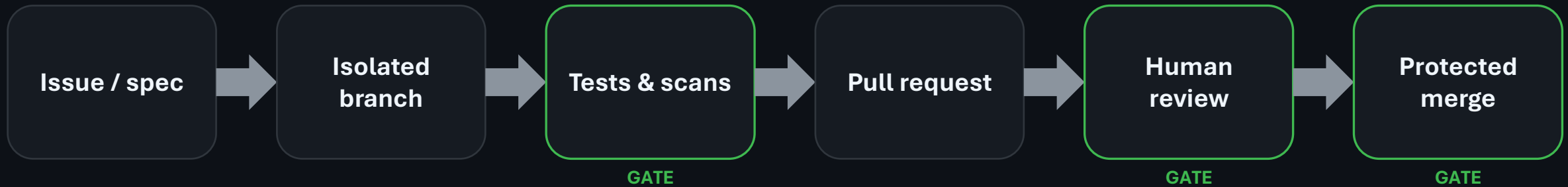


As autonomy rises, clarity, tests, permissions, and review must rise too.

Use Plan before execution. “Autopilot” is an app mode, not a universal Copilot mode.

GitHub is the control plane

Agentic work runs through the same SDLC controls you already trust.



Session log / audit trail — traceability and intervention points at every step

Treat agent output as contributor output.

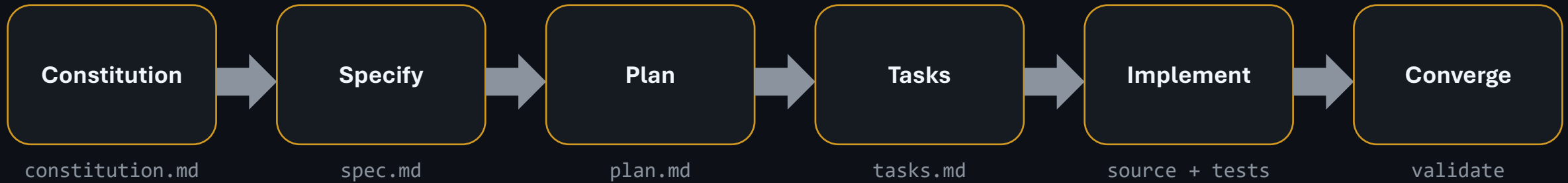
The cloud agent is branch- and review-bounded. Generated code still needs validation. You own the merge.

Security features are scoped controls, not a guarantee that all generated code is safe.

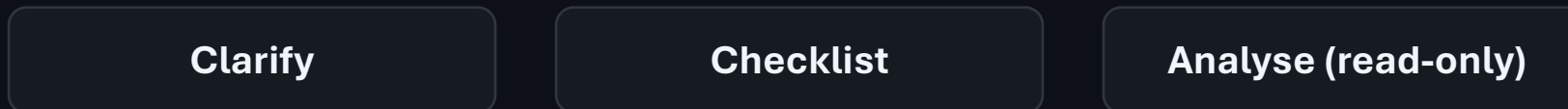
SPECIFY

Agentic Engineering with Spec Kit

Durable artifacts manage work that outlives a single prompt.



QUALITY GATES



Specify = what & why · Plan = how (technology) · Tasks preserve dependency order

Analyse checks consistency without editing source. Converge checks the build against intent.

Supports — does not replace — normal engineering governance.

Your homework

New

Ask about one unfamiliar function.

Generate and inspect one test.

Regular

Improve repository instructions.

Use Plan before the next medium change.

Advanced

Package one repeated workflow.

Review available MCP tools.
Measure rework.

Better context beats a more expensive model.

More autonomy requires clearer specifications and stronger controls.

You remain accountable.

Awesome Copilot Hub

github.com/github/awesome-copilot

GH-300 study guide

Microsoft Learn · GH-300

GH-600 study guide

Microsoft Learn · GH-600

Pick one card. Progress means changing a work habit, not enabling every feature.